EXACT BYTE-LEVEL PROBABILITIES FROM TOKENIZED LANGUAGE MODELS

Anonymous authors
Paper under double-blind review

ABSTRACT

Tokenization makes modern language models (LMs) efficient, but it also changes what they predict. Prompting a tokenized LM means conditioning on one particular encoding of the context. The prompt implicitly asserts that a token boundary falls where the prompt ends, so the predictive distribution can differ systematically from the byte-level distribution of the text. We call this effect *tokenization bias*. It is most damaging when a prompt ends in the middle of a token, as in fill-in-the-middle (FIM) code completion: the correct next character can then receive exactly zero probability. We show that a tokenized LM still determines an equivalent byte-level process, which can be recovered without retraining. Our *Byte-Token Representation Lemma* writes the probability of any byte string as a sum over a small set of *cover encodings*. These are found by a backward search that uses the tokenizer to discard invalid encodings. On top of this lemma, we build an inference-time sampler that converts next-token predictions into exact next-byte predictions with a single model call per byte. On synthetic Markov chains, the method recovers the true transition probabilities that the naive tokenized predictor distorts. On the HumanEval random-span FIM benchmark with CodeLlama-7b, it raises the greedy pass rate of the mid-token Suffix-Prefix-Middle format from 45.0% to 63.9% (about 18 points). This also beats the token healing and token alignment heuristics. Mapping models into a shared byte space also lets us ensemble LMs that use different tokenizers. A simple average of the top two models outperforms the best single model on most benchmarks, by up to 3.7 points on HumanEval. It does not help when the member models differ widely in capability.

1 INTRODUCTION

Almost every state-of-the-art language model operates on *tokens* rather than on bytes or characters (Radford et al., 2019; Brown et al., 2020; Touvron et al., 2023; Jiang et al., 2023a). Subword tokenizers such as Byte-Pair Encoding (BPE) (Sennrich et al., 2016), WordPiece (Schuster & Nakajima, 2012) and SentencePiece (Kudo & Richardson, 2018) compress text into shorter sequences. Shorter sequences reduce the cost of Transformer training and inference (Vaswani et al., 2017), and token-level modeling still clearly outperforms byte-level modeling at a fixed compute budget. Tokenization is therefore unlikely to disappear soon. Yet it is a lossy interface between the user, who thinks in characters, and the model, which thinks in tokens.

This paper studies a statistical consequence of that interface. A tokenized LM is trained on sequences produced by a deterministic tokenizer, so some token sequences never occur in training. For instance, under BPE two adjacent tokens are never followed by a pair that the merge rules would have combined. At inference time the user supplies a byte string, which is tokenized and fed to the model. The next-token distribution then conditions not only on the text, but also on the event that *the tokenization of the full text keeps a token boundary exactly at the end of the prompt*. This event carries information about the future. As a result, the model's predictive distribution over the next byte can differ from the true conditional distribution of the next byte given the prompt. We call this discrepancy *tokenization bias*. It is not a modeling error that more data or capacity would remove: it persists even for a model that matches the training distribution perfectly.

Figure 1 (left) shows a practical case. A code completion prompt ends in the middle of a token, as happens in fill-in-the-middle (FIM) tasks (Bavarian et al., 2022; Fried et al., 2023). Because

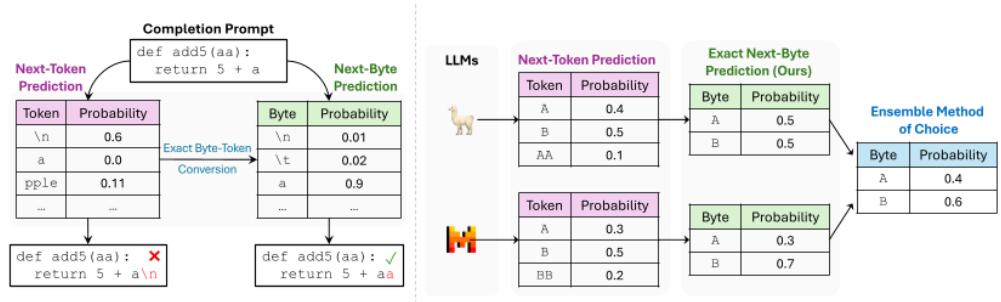


Figure 1: **Left:** tokenized LMs can suffer tokenization bias when prompts end mid-token, as in this code completion example: the correct continuation `a` has zero probability of being chosen. Our method avoids this problem and predicts the correct byte with the same model. **Right:** our method maps the next-token predictions of arbitrary tokenized LMs to statistically equivalent next-byte predictions (Section 3). This enables any ensembling strategy, such as averaging or mixture of experts.

the tokenizer would have merged the final characters with the correct continuation, the correct next character has *zero* probability under the tokenized model. Practitioners have proposed heuristics such as *token healing* (Dagan et al., 2024) and *token alignment* (Athiwaratkun et al., 2024). These back up a few tokens and constrain generation to match the removed characters. They help, but they lack a precise statistical justification.

We take a principled route. Tokenization bias comes from conditioning on a single deterministic encoding of the past. If we instead *marginalize* over all token sequences that are consistent with the observed bytes, the bias disappears. Naively, this sum ranges over exponentially many tokenizations. We show, however, that deterministic tokenizers make most tokenizations *invalid*: they can never be produced by the tokenizer and thus carry zero probability. Only a small set of *cover encodings* remains. These are valid token sequences whose last token straddles the end of the byte string. This observation yields an exact and efficient conversion from token-level to byte-level probabilities that needs no retraining.

Contributions.

- **Theory.** We formalize tokenization bias and show that it affects even a perfectly trained model (Proposition 1). We prove a *Byte-Token Representation Lemma* (Lemma 1), which expresses the byte-level probability of any string exactly in terms of token-level probabilities of its cover encodings.
- **Algorithms.** We give a backward search that enumerates cover encodings with a bounded number of candidates (Algorithm 1). We also give a next-byte sampler that reuses cover encodings across steps and needs one model call per generated byte (Algorithm 2). Together they turn any tokenized LM into a token-free LM at inference time.
- **Robust FIM.** On HumanEval random-span infilling with CodeLlama-7b, byte-level prediction removes most of the degradation caused by mid-token prompts. It improves greedy pass rates in the Suffix-Prefix-Middle format by about 18 points and outperforms token healing and token alignment.
- **Ensembling across vocabularies.** Byte-level predictions give heterogeneous LMs a common output space. A simple average of the top two models improves over the best individual model on most knowledge, reasoning and code benchmarks we evaluate. We also identify when this fails.

2 Related Work

Tokenization. Subword tokenization was popularized in neural machine translation by BPE (Sennrich et al., 2016) and WordPiece (Schuster & Nakajima, 2012). It is now standard via libraries

such as SentencePiece (Kudo & Richardson, 2018); see Mielke et al. (2021) for a survey. Its downsides are well documented. Different tokenizers compress text with different efficiency (Bostrom & Durrett, 2020; Rust et al., 2021), tokenization can introduce disparities across languages (Petrov et al., 2023), and tokenizer choices affect downstream behavior (Dagan et al., 2024). Stochastic tokenization at training time, such as subword regularization (Kudo, 2018) and BPE-dropout (Provilkov et al., 2020), exposes models to several segmentations of the same string. By contrast, we leave training untouched and correct the predictions of models trained with deterministic tokenizers. Minixhofer et al. (2024) transfer models to new tokenizers through a learned hypernetwork. Our approach instead requires no learning and maps every model to the same byte-level space.

Theory of tokenization. Recent work studies tokenization formally. Zouhar et al. (2023) relate tokenizer quality to information-theoretic efficiency, and Gastaldi et al. (2024) give a formal framework for the statistical consistency of tokenizer-based estimators. Rajaraman et al. (2024) analyze Transformers trained on k-th order Markov processes with and without tokenization. Like them, we use Markov chains as an analytically tractable testbed. Our focus differs: we study the mismatch between token-conditional and byte-conditional predictions at inference time, and how to remove it exactly.

Marginalizing over tokenizations. Cao & Rimell (2021) argue that LMs should be evaluated by the marginal likelihood over all tokenizations of a string, which they estimate by sampling. In psycholinguistics, Pimentel & Meister (2024) and Oh & Schuler (2024) show that word probabilities computed naively from subword LMs are incorrect, notably because of leading whitespace in tokens, and derive corrections. Our Byte-Token Representation Lemma generalizes this line of work to arbitrary byte strings. It exploits the deterministic structure of the tokenizer to restrict the marginalization to a small set of cover encodings, which makes *exact* computation and *autoregressive sampling* practical.

Prompt boundary heuristics. *Token healing* (Dagan et al., 2024) removes the last token of the prompt and constrains the first generated token to begin with the removed characters. *Token alignment* (Athiwaratkun et al., 2024) backtracks several tokens and constrains generation to match the removed prefix. Both mitigate the mid-token problem in code completion. However, they do not guarantee the correct conditional distribution: they still condition on one particular tokenization of the shortened prompt. Our method instead computes exact byte-level conditionals, and we compare against both heuristics directly.

Byte-level and token-free models. Token-free models operate directly on bytes or characters. Examples include ByT5 (Xue et al., 2022), CANINE (Clark et al., 2022), Charformer (Tay et al., 2022), MEGABYTE (Yu et al., 2023) and MambaByte (Wang et al., 2024). These avoid tokenization bias by construction but require training new models, often at higher cost. Our method obtains a token-free model from any existing tokenized LM at inference time.

Ensembling LMs. Ensembling is a classical way to combine complementary models (Dietterich, 2000; Jacobs et al., 1991; Shazeer et al., 2017). For LLMs, differing vocabularies prevent direct averaging of next-token distributions. Existing approaches therefore operate on complete outputs, as in voting and self-consistency (Wang et al., 2023) or ranking and fusion (Jiang et al., 2023b). Others distill several models into one (Wan et al., 2024), or align vocabularies through learned or heuristic mappings, such as embedding projections (Xu et al., 2024), relative representations (Huang et al., 2024) or vocabulary unions (Yu et al., 2024). Our byte-level conversion gives a mapping into a shared space that is *exact* under the model and requires neither training nor vocabulary matching.

3 METHODOLOGY

3.1 PRELIMINARIES

Let $\mathcal{A}$ be a finite alphabet of *bytes*. We use "byte" for the atomic symbol of the tokenizer's base alphabet. We write $x_i^j = x_i x_{i+1} \cdots x_j$ for a string, with x_i^j empty if $j < i$. A tokenizer is given by a vocabulary $\mathcal{V} \subset \mathcal{A}^+$ with $\mathcal{A} \subseteq \mathcal{V}$, so every byte is a token. It also has a pair of maps $\mathrm{decode} : \mathcal{V}^* \to \mathcal{A}^*$, which concatenates tokens, and $\mathrm{encode} : \mathcal{A}^* \to \mathcal{V}^*$, which is deterministic and satisfies $\mathrm{decode}(\mathrm{encode}(\boldsymbol{x})) = \boldsymbol{x}$. We write $\boldsymbol{t} = t_1^k$ for token sequences, $\boldsymbol{t} \oplus v$ for appending token v, and $L = \max_{v \in \mathcal{V}} |v|$ for the maximum token length.

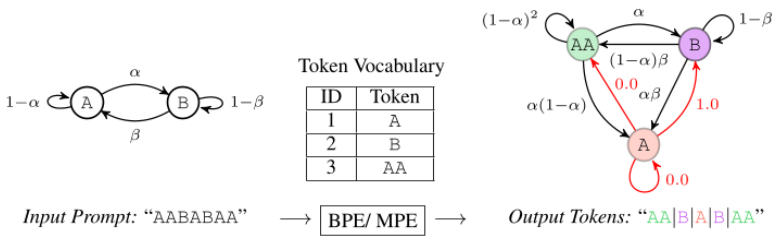


Figure 2: Tokenization bias on a 1st-order Markov chain. Given the context token "A", the model never samples A as the next token, whereas the true probability of the next byte being A is $1 - \alpha$. In practice this bias occurs when prompts end with tokens that are part of another token, a common issue in FIM tasks, and it leads to incorrect completions.

Byte and token processes. Text is generated by a byte-level stochastic process $\mathrm{X} = (\mathrm{X}_1, \mathrm{X}_2, \dots)$ with prefix probabilities $P(x_1^n) := \Pr(\mathrm{X}_1^n = x_1^n)$. A tokenized LM is trained on the token process $\mathrm{T} = \mathrm{encode}(\mathrm{X})$ and models its prefix probabilities $P(t_1^k) := \Pr(\mathrm{T}_1^k = t_1^k) = \prod_{i=1}^k P(t_i \mid t_1^{i-1})$. For the analysis we assume the LM matches the token process exactly. This is the most favorable case for the tokenized model, and it shows that the issues we identify are not due to estimation error. We require one structural property of the tokenizer.

Assumption 1 (Prefix consistency)**.** *For any string $\boldsymbol{x}$, finite or infinite, and any k, the first k tokens t_1^k of* $\mathrm{encode}(\boldsymbol{x})$ *satisfy* $\mathrm{encode}(\mathrm{decode}(t_1^k)) = t_1^k$.

In words, re-encoding the text covered by any prefix of a tokenization returns that same prefix. The assumption holds exactly for maximum prefix encoding (MPE, greedy longest-match) tokenizers. It holds for BPE whenever later bytes cannot change earlier merges, which is the overwhelming majority of cases in practice, especially with pre-tokenization. It makes the token process well defined and leads to the central notion of validity.

Definition 1 (Valid and invalid encodings)**.** *A token sequence $\boldsymbol{t} \in \mathcal{V}^*$ is* valid *if* $\mathrm{encode}(\mathrm{decode}(\boldsymbol{t})) = \boldsymbol{t}$*, and* invalid *otherwise.*

By Assumption 1, every prefix of T is valid, so $P(\boldsymbol{t}) = 0$ for every invalid $\boldsymbol{t}$. For example, with $\mathcal{V} \supseteq \{\mathtt{A}, \mathtt{AA}\}$ under MPE, the sequence $(\mathtt{A}, \mathtt{A})$ is invalid because $\mathrm{encode}(\mathtt{AA}) = (\mathtt{AA})$. A trained LM never observes such sequences. It may assign them some probability, but this mass reflects neither the data nor the byte process.

3.2 Tokenization Bias

The standard way to predict the next byte with a tokenized LM is to encode the prompt, $t_1^k = \mathrm{encode}(x_1^n)$, and read off the next-token distribution:

$$\hat{P}(x_{n+1} = a \mid x_1^n) \;=\; \sum_{v \in \mathcal{V}:\, v_1 = a} P(t_{k+1} = v \mid t_1^k) \;=\; \Pr\big(\mathrm{X}_{n+1} = a \,\big|\, \mathrm{T}_1^k = \mathrm{encode}(x_1^n)\big). \quad (1)$$

The event $\{\mathrm{T}_1^k = \mathrm{encode}(x_1^n)\}$ is strictly smaller than $\{\mathrm{X}_1^n = x_1^n\}$. It additionally asserts that the tokenization of the *entire* text has a token boundary after position n, which constrains the bytes that follow. We define the *tokenization bias* as the discrepancy between $\hat{P}(x_{n+1} \mid x_1^n)$ and $P(x_{n+1} \mid x_1^n)$. The next proposition shows that it can be as large as possible, even for a perfect model.

Proposition 1 (Tokenization bias under a perfect model)**.** *Consider a 1st-order Markov chain on $\mathcal{A} = \{\mathtt{A}, \mathtt{B}\}$ with $P(\mathtt{B} \mid \mathtt{A}) = \alpha$ and $P(\mathtt{A} \mid \mathtt{B}) = \beta$, where $\alpha, \beta \in (0, 1)$. Tokenize it with MPE and $\mathcal{V} = \{\mathtt{A}, \mathtt{B}, \mathtt{AA}\}$ (Figure 2). Then for every prompt x_1^n whose encoding ends with the token $\mathtt{A}$, the naive predictor gives $\hat{P}(x_{n+1} = \mathtt{A} \mid x_1^n) = 0$, while $P(x_{n+1} = \mathtt{A} \mid x_1^n) = 1 - \alpha$.*

Proof. If the token A at the end of $\mathrm{encode}(x_1^n)$ were followed by the byte A in the full text, MPE would have matched the longer token AA at that position. Hence $\Pr(\mathrm{X}_{n+1} = \mathtt{A} \mid \mathrm{T}_1^k = \mathrm{encode}(x_1^n)) = 0$. The true conditional follows from the Markov property. □

The same mechanism underlies the FIM failure in Figure 1. Once the prompt ends with a token that is a proper prefix of other tokens, every continuation that the tokenizer would have merged becomes impossible. Increasing model capacity or data cannot fix this, because the bias is a property of the *conditioning event*, not of the estimator.

3.3 The Byte-Token Representation Lemma

To remove the bias, we compute $P(x_1^n)$, and hence $P(x_{n+1} \mid x_1^n) = P(x_1^{n+1})/P(x_1^n)$, from token probabilities. The relevant token sequences are those whose last token covers the final byte of the string.

Definition 2 (Cover encodings)**.** *The set of* cover encodings *of x_1^n (with $n \geq 1$) is*

$$\mathrm{cover}(x_1^n) = \left\{ t_1^k \textit{ valid } : \; x_1^n \preceq \mathrm{decode}(t_1^k) \textit{ and } \mathrm{decode}(t_1^{k-1}) \prec x_1^n \right\},$$

where $\boldsymbol{y} \preceq \boldsymbol{z}$ denotes that $\boldsymbol{y}$ is a prefix of $\boldsymbol{z}$ and $\boldsymbol{y} \prec \boldsymbol{z}$ that it is a proper prefix.

So the last token t_k starts at some position $i \leq n$ and reaches at least to position n, possibly extending beyond it (Figure 3, left).

Lemma 1 (Byte-Token Representation)**.** *Under Assumption 1, for every byte string x_1^n,*

$$P(x_1^n) \;=\; \sum_{\boldsymbol{t} \in \mathrm{cover}(x_1^n)} P(\boldsymbol{t}). \tag{2}$$

Proof sketch. For any realization of X beginning with x_1^n, exactly one token of T contains position n; the prefix of T ending with that token is valid (Assumption 1) and lies in $\mathrm{cover}(x_1^n)$, and no other cover encoding can be a prefix of T. Hence the events $\{\mathrm{T}_1^{|\boldsymbol{t}|} = \boldsymbol{t}\}$, $\boldsymbol{t} \in \mathrm{cover}(x_1^n)$, partition $\{\mathrm{X}_1^n = x_1^n\}$ (full proof in Appendix A). □

The cover set is small because the history before the last token is forced.

Lemma 2 (Structure of cover encodings)**.** *Under Assumption 1, every $\boldsymbol{t} \in \mathrm{cover}(x_1^n)$ has the form $\boldsymbol{t} = \mathrm{encode}(x_1^{i-1}) \oplus v$, where $i \in \{\max(1, n-L+1), \ldots, n\}$ and $v \in \mathcal{V}$ has x_i^n as a prefix. Consequently, $|\mathrm{cover}(x_1^n)| \leq \sum_{i=\max(1,n-L+1)}^{n} \left|\{v \in \mathcal{V} : x_i^n \textit{ is a prefix of } v\}\right|$.*

Proof. Since $\boldsymbol{t}$ is valid, $\boldsymbol{t} = \mathrm{encode}(\mathrm{decode}(\boldsymbol{t}))$. Assumption 1 then implies that t_1^{k-1} is valid, so $t_1^{k-1} = \mathrm{encode}(\mathrm{decode}(t_1^{k-1})) = \mathrm{encode}(x_1^{i-1})$, where $i - 1 = |\mathrm{decode}(t_1^{k-1})| < n$. The last token must contain x_i^n as a prefix, which forces $n - i + 1 \leq L$. □

Lemmas 1 and 2 convert an apparently exponential marginalization into a sum with at most L groups of candidates. Each group requires the next-token distribution after a single, deterministic context $\mathrm{encode}(x_1^{i-1})$, because $P(\mathrm{encode}(x_1^{i-1}) \oplus v) = P(\mathrm{encode}(x_1^{i-1}))\, P(v \mid \mathrm{encode}(x_1^{i-1}))$.

3.4 Backward Search for Cover Encodings

Lemma 2 suggests a *backward* search (Algorithm 1, Figure 3 right). We move the candidate start position i of the last token from n down to $n - L + 1$. For each i, we encode the history x_1^{i-1} once, enumerate the tokens v that begin with x_i^n, and keep $\mathrm{encode}(x_1^{i-1}) \oplus v$ only if it is valid, i.e., if re-encoding its decoded string reproduces it. The search never revisits the context beyond the last L bytes, apart from computing $\mathrm{encode}(x_1^{i-1})$ and its prefix probability. The latter can share a key-value cache across iterations because the encodings differ only near the end. Prefix tries over $\mathcal{V}$ make the enumeration of tokens with a given prefix efficient.

3.5 Exact Next-Byte Sampling

For generation, we need $P(x_{n+1} = a \mid x_1^n)$ for all $a \in \mathcal{A}$ at every step. Rerunning Algorithm 1 for each candidate byte would be wasteful. Instead, we maintain $\mathrm{cover}(x_1^n)$ with probabilities and update it incrementally (Figure 4). Let $\boldsymbol{\tau}_n = \mathrm{encode}(x_1^n)$. Note that $\boldsymbol{\tau}_n \in \mathrm{cover}(x_1^n)$: it is the unique cover encoding whose last token ends exactly at position n, so $P(\boldsymbol{\tau}_n)$ is already known.

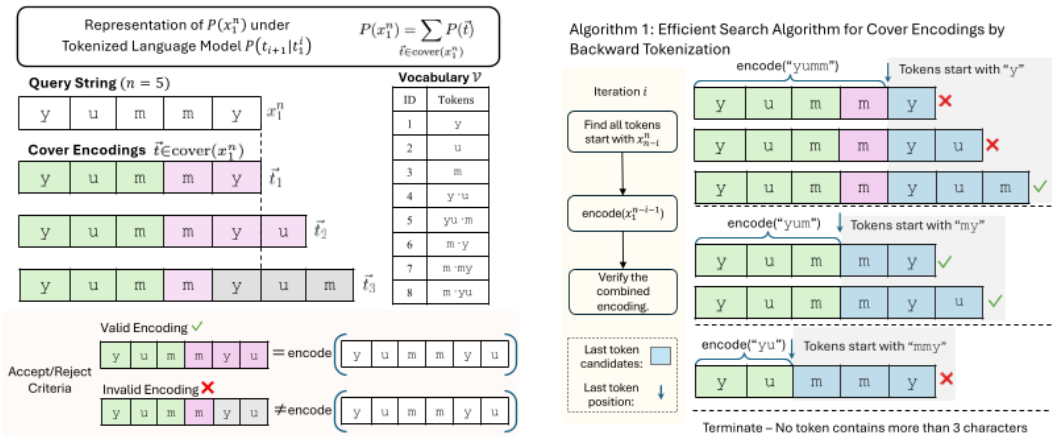


Figure 3: **Left:** representation of $P(x_1^n)$ under a tokenized LM, with an example of cover encodings and of valid/invalid encodings. **Right:** illustration of Algorithm 1 for cover encoding search. A green tick and a red cross denote valid and invalid encodings (Definition 1). The termination step (no token is longer than the remaining suffix) is easy to implement although not shown in the algorithm. We use BPE in this example.

Algorithm 1 Backward search for cover encodings

Require: byte string x_1^n, tokenizer $(\text{encode}, \text{decode}, \mathcal{V})$, tokenized LM P

$\mathcal{C} \leftarrow \emptyset$
for $i = n, n-1, \ldots, \max(1, n-L+1)$ **do**
  $\boldsymbol{\tau} \leftarrow \text{encode}(x_1^{i-1})$; compute $P(\boldsymbol{\tau})$ and $P(\cdot \mid \boldsymbol{\tau})$ ▷ one model call
  for each $v \in \mathcal{V}$ such that x_i^n is a prefix of v **do**
    if $\text{encode}(\text{decode}(\boldsymbol{\tau})\, v) = \boldsymbol{\tau} \oplus v$ **then** ▷ validity check (Def. 1)
      $\mathcal{C} \leftarrow \mathcal{C} \cup \{(\boldsymbol{\tau} \oplus v,\ P(\boldsymbol{\tau})\, P(v \mid \boldsymbol{\tau}))\}$
    end if
  end for
end for
return $\mathcal{C}$ ▷ $P(x_1^n) = \sum_{(\boldsymbol{t},p) \in \mathcal{C}} p$ by Lemma 1

Proposition 2 (Incremental decomposition)**.** *Under Assumption 1, for every* $a \in \mathcal{A}$, $\text{cover}(x_1^n a) = \mathcal{C}_{n+1}(a) \cup \bar{\mathcal{C}}_{n+1}(a)$ *(disjoint), where*

$$\mathcal{C}_{n+1}(a) = \{\boldsymbol{t} \in \text{cover}(x_1^n) : |\text{decode}(\boldsymbol{t})| > n,\ \text{decode}(\boldsymbol{t})_{n+1} = a\},$$
$$\bar{\mathcal{C}}_{n+1}(a) = \{\boldsymbol{\tau}_n \oplus v : v \in \mathcal{V},\ v_1 = a,\ \boldsymbol{\tau}_n \oplus v \textit{ valid}\}.$$

Hence

$$P(x_1^n a) \;=\; \sum_{\boldsymbol{t} \in \mathcal{C}_{n+1}(a)} P(\boldsymbol{t}) \;+\; P(\boldsymbol{\tau}_n) \sum_{v:\, \boldsymbol{\tau}_n \oplus v \,\in\, \bar{\mathcal{C}}_{n+1}(a)} P(v \mid \boldsymbol{\tau}_n). \tag{3}$$

The first set contains the cover encodings of x_1^n whose last token already extends past position n with byte a at position $n+1$ (*regrouping*). The second contains new encodings whose last token starts at position $n+1$ (*extension*). By Lemma 2, the history of an extension must be exactly $\boldsymbol{\tau}_n$. The proof is in Appendix A. Eq. (3) yields Algorithm 2 (Appendix B). Each step needs a single model call, namely $P(\cdot \mid \boldsymbol{\tau}_n)$, together with tokenizer-only validity checks. After sampling x_{n+1}, the set $\text{cover}(x_1^{n+1}) = \mathcal{C}_{n+1}(x_{n+1}) \cup \bar{\mathcal{C}}_{n+1}(x_{n+1})$ is directly available for the next step. Algorithm 1 is therefore run only once, on the prompt.

Normalization with imperfect models. For an exact model, $\sum_a P(x_1^n a) = P(x_1^n)$, since the byte process continues with probability one. A trained LM assigns some mass to invalid encodings, and Eq. (3) discards it. We therefore normalize over the alphabet: $P(x_{n+1} = a \mid x_1^n) \propto P(x_1^n a)$. Discarding invalid encodings is a feature of the method: the tokenizer, not the model, decides which encodings can occur.

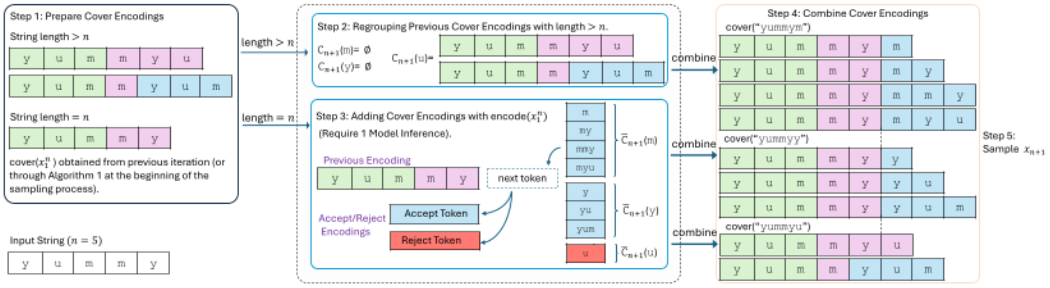


Figure 4: Illustration of sampling x_{n+1} from a tokenized LM $P(t_{k+1} \mid t_1^k)$, continuing the example of Figure 3. Step 2 regroups previous cover encodings that extend past position n ($\mathcal{C}_{n+1}$); Step 3 adds new encodings after $\text{encode}(x_1^n)$ using one model call ($\bar{\mathcal{C}}_{n+1}$); Step 4 combines them into $\text{cover}(x_1^n a)$ for each byte a. We omit $P(\boldsymbol{t})$ for simplicity. Details in Section 3.5.

3.6 Applications

Fill-in-the-middle and special tokens. FIM prompts interleave special sentinel tokens with code. Sentinels are never merged with ordinary bytes, so they form hard token boundaries. We keep everything up to the last sentinel as a fixed token context and apply the byte-level conversion to the byte string that follows it. In the Prefix-Suffix-Middle (PSM) format (Bavarian et al., 2022), generation starts right after a sentinel. In the Suffix-Prefix-Middle (SPM) format, the generated middle directly continues the code prefix, which in the random-span setting typically ends mid-token. This is exactly where tokenization bias arises.

Byte-level decoding and beam search. Greedy byte-level decoding selects $\arg\max_a P(x_{n+1} = a \mid x_1^n)$. This marginalizes over all tokens that share a first byte, which differs from greedy token-level decoding. Byte-level beam search keeps the B byte prefixes with the highest $P(x_1^{n+1})$. Each beam carries its own cover set, and the joint probabilities follow from Lemma 1.

Ensembling across vocabularies. Each tokenized model P_m, with its own tokenizer, induces a distribution $P_m(x_{n+1} \mid x_1^n)$ over the *same* alphabet. We combine M models by averaging in byte space, $P_{\text{ens}}(x_{n+1} \mid x_1^n) = \frac{1}{M}\sum_{m=1}^{M} P_m(x_{n+1} \mid x_1^n)$, and decode from P_{ens} with Algorithm 2. Each member maintains its own cover set. More elaborate combination rules, such as weighted averages or mixtures of experts (Jacobs et al., 1991), plug in identically (Figure 1, right).

4 Experiments

We evaluate three questions. (i) Does tokenization bias occur as predicted, and does our conversion remove it (Section 4.1)? (ii) Does byte-level prediction fix mid-token prompts in code infilling (Section 4.2)? (iii) How do byte-level predictions and byte-level ensembles perform on standard benchmarks (Section 4.3)?

Models. We use general-purpose LMs (Llama2-7B (Touvron et al., 2023), Yi-1.5-6B (Young et al., 2024), Mistral-7B-v0.3 (Jiang et al., 2023a)) and code LMs (CodeLlama-7b and -13b (Rozière et al., 2023), Yi-Coder-1.5B). All are Transformer LLMs with different tokenizers and vocabularies. For the synthetic study we train a GPT-2-style Transformer (Radford et al., 2019) with 6 hidden layers on tokenized Markov chain data.

Benchmarks and metrics. For infilling we use the HumanEval random-span benchmark (Bavarian et al., 2022), reporting pass rates with greedy decoding and pass@10. For general LMs we use MMLU (Hendrycks et al., 2021), PIQA (Bisk et al., 2020), Natural Questions (NQ) (Kwiatkowski et al., 2019), TriviaQA (Joshi et al., 2017) and GSM8K (Cobbe et al., 2021), reporting accuracy. For code generation we use HumanEval (Chen et al., 2021) and MBPP (Austin et al., 2021), reporting pass@1. In each case we compare the standard *token-level* predictions with *byte-level* predictions from our method, applied to the same model without any weight updates.

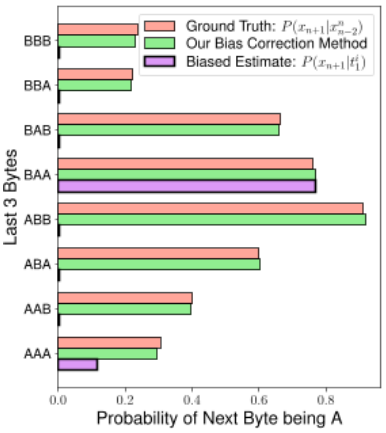


Figure 5: Tokenization bias on a 3rd-order Markov chain. For each context of three bytes, we show the probability that the next byte is A under the ground-truth chain, under the naive (biased) estimate $P(x_{n+1} \mid t_1^k)$ of a trained tokenized model, and under our byte-token conversion (bias correction, Section 3) applied to the same model. Our method accurately recovers $P(x_{n+1} \mid x_{n-2}^n)$ of the original chain.

Predictions	PSM Greedy	PSM @10	SPM Greedy	SPM @10
Token-level	64.9	84.0	45.0	66.5
+ healing	|	|	62.7	83.8
+ alignment	|	|	63.0	82.9
Byte-level	**65.7**	**84.1**	**63.9**	**84.3**

@10 here is the max over the temperature sweep on the right

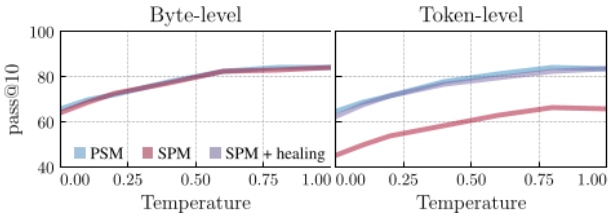


Figure 6: Pass rates on the HumanEval random-span benchmark (Bavarian et al., 2022) with CodeLlama-7b for infilling Python. Across all settings, byte-level predictions yield better completions. We evaluate the two standard infilling templates, Prefix-Suffix-Middle (PSM) and Suffix-Prefix-Middle (SPM), and use token healing (Dagan et al., 2024) and token alignment (Athiwaratkun et al., 2024) as baselines. Left: greedy pass rates and pass@10, where pass@10 is the maximum over the temperature sweep. Right: pass@10 as a function of sampling temperature. The SPM template forces the model to predict from the middle of a token, which byte-level predictions handle.

Baselines. Besides standard token-level prediction, we compare against *token healing* (Dagan et al., 2024) and *token alignment* (Athiwaratkun et al., 2024) in the SPM infilling setting, where prompt boundaries are problematic. For ensembling, we compare against the individual members and, for multiple-choice tasks, a top-2 voting baseline. Ensembles always combine the two best-performing models on the respective benchmark with uniform weights in byte space.

4.1 Tokenization Bias on Markov Chains

On the 1st-order chain of Figure 2, the tokenized model behaves exactly as Proposition 1 predicts. After a context ending in the token A, the next token is never A: the probability of the byte A following the context A is 0.0 in token space, whereas the ground truth is $1 - \alpha$.

We then train the 6-layer GPT-2 model on tokenized samples from a 3rd-order Markov chain over $\{\mathtt{A}, \mathtt{B}\}$. Figure 5 compares, for each 3-byte context, the true transition probability $P(x_{n+1} = \mathtt{A} \mid x_{n-2}^n)$, the naive estimate conditioned on the encoded context, and our byte-level estimate computed from the same trained model. The naive predictor is severely biased: for most contexts it assigns near-zero probability to the next byte being A, even when the true probability is large. Our bias-corrected estimate closely matches the ground truth for every context. This confirms that the trained tokenized model already contains the byte-level process, and that Lemma 1 recovers it.

4.2 Fill-in-the-Middle Code Completion

Figure 6 reports infilling results for CodeLlama-7b.

Mid-token prompts severely degrade token-level models. With standard token-level prediction, switching from PSM to SPM lowers the greedy pass rate from 64.9% to 45.0% and pass@10 from

Table 1: Accuracy (%) of general-purpose LMs with token-level (Tok.) and byte-level (Byte) predictions, and of top-2 voting and our top-2 byte-level ensemble. N/A: not applicable (ensembles and voting only exist in byte space; voting is only applicable to multiple-choice tasks). Bold marks the best result per benchmark.

	MMLU		PIQA		NQ		TriviaQA		GSM8K	
Model	Tok.	Byte	Tok.	Byte	Tok.	Byte	Tok.	Byte	Tok.	Byte
Llama2-7B	45.4	45.7	78.1	78.2	25.0	23.4	58.4	58.1	15.1	12.5
Yi-1.5-6B	63.4	63.4	78.5	78.5	22.8	22.7	53.7	53.4	61.3	**61.5**
Mistral-7B-v0.3	62.1	62.1	80.1	80.3	28.5	28.8	63.6	63.5	39.0	39.0
Voting (top-2)	N/A	62.1	N/A	80.1	N/A	N/A	N/A	N/A	N/A	N/A
Top-2 ensemble (ours)	N/A	**65.4**	N/A	**80.7**	N/A	**30.0**	N/A	**64.2**	N/A	55.8

84.0% to 66.5%. The SPM curve also stays well below PSM across the whole temperature sweep (Figure 6, right). In SPM the code prefix immediately precedes the generated middle, and random spans frequently cut words in half. These out-of-distribution token boundaries are exactly the setting of Proposition 1.

Byte-level prediction recovers PSM-level performance. Applied to the same model, our method raises SPM greedy to 63.9% and pass@10 to 84.3%. This is an improvement of 18.9 and 17.8 points over token-level prediction, and comparable to PSM performance. The byte-level SPM and PSM curves nearly coincide over the whole temperature sweep, which suggests that the conversion effectively removes the dependence on where the prompt ends.

Byte-level prediction outperforms heuristic fixes. Token healing (62.7% greedy, 83.8% pass@10) and token alignment (63.0%, 82.9%) recover much of the SPM gap, but both remain below byte-level prediction (63.9%, 84.3%). Unlike these heuristics, our method does not commit to a single tokenization of a shortened prompt. It sums over all valid encodings that are consistent with the bytes.

Gains even when boundaries are benign. In PSM, where generation starts right after a sentinel, byte-level prediction still improves greedy decoding from 64.9% to 65.7% (pass@10: 84.0% vs. 84.1%). We attribute this to two effects. First, random spans can still end mid-token relative to the suffix that follows. Second, greedy byte-level decoding aggregates probability over all tokens that share the next byte instead of committing to the single most likely token. The magnitude of this effect is small, and we do not claim it generalizes beyond this setting.

Qualitative analysis. Inspecting failures of the token-level SPM model, we find many cases where the prompt ends mid-word, for example within identifiers such as `delimeter`, `collatz` and `palindrome`. The token-level model fails to complete the word correctly because of the token boundary mismatch, as in Figure 1 (left). The byte-level method identifies the correct completion.

Ablation: beam width. Table 3 varies the width of byte-level beam search. Wider beams yield consistent gains: from width 1 to 4, PSM improves from 65.7% to 67.7% and SPM from 63.9% to 66.8%, a gain of up to 2.9 points.

4.3 Byte-Level Predictions and Cross-Vocabulary Ensembles

Byte-level vs. token-level predictions. Tables 1 and 2 show that on standard benchmarks, where prompts end at natural boundaries, byte-level and token-level predictions usually perform similarly. On MMLU, PIQA, NQ and TriviaQA most differences are within a few tenths of a point, in both directions. The picture is less uniform on generation-heavy tasks. Byte-level prediction lowers Llama2-7B on GSM8K (15.1 to 12.5) and NQ (25.0 to 23.4), and lowers all three code models on HumanEval (by 1.9 to 3.6 points). On MBPP it improves CodeLlama-7b (40.6 to 42.4) and Yi-Coder-1.5B (52.8 to 53.6). We do not have a definitive explanation for the HumanEval drops. Plausible factors include the following. Byte-level greedy decoding is not equivalent to token-level greedy decoding, which matches the decoding procedure most commonly used to evaluate these

Table 2: Pass@1 (%) of code LMs with token-level (Tok.) and byte-level (Byte) predictions, and of our top-2 byte-level ensemble. Bold: best per benchmark.

	HumanEval		MBPP	
Model	Tok.	Byte	Tok.	Byte
CodeLlama-7b	32.3	28.7	40.6	42.4
CodeLlama-13b	35.8	33.5	47.6	47.4
Yi-Coder-1.5B	38.4	36.5	52.8	**53.6**
Top-2 ensemble (ours)	N/A	**42.1**	N/A	**53.6**

Table 3: Ablation on beam width for byte-level FIM decoding with CodeLlama-7b (HumanEval random-span pass rate, %). Width 1 is greedy byte-level decoding.

Beam width	1	2	4
PSM	65.7	66.8	**67.7**
SPM	63.9	65.9	**66.8**

models. Trained models also place non-negligible mass on invalid encodings, which our method discards and renormalizes. Finally, small per-step differences can compound over long generations. In this regime the main benefit of the byte-level view is not single-model accuracy but the common output space it provides.

Ensembling models with different vocabularies. Llama2, Yi and Mistral use different tokenizers, so their next-token distributions cannot be averaged directly. In byte space they can. Our top-2 ensemble outperforms the best individual model on MMLU (65.4 vs. 63.4), PIQA (80.7 vs. 80.3), NQ (30.0 vs. 28.8) and TriviaQA (64.2 vs. 63.6). On the multiple-choice tasks it also clearly beats top-2 voting (62.1 on MMLU, 80.1 on PIQA). For code, ensembling Yi-Coder-1.5B and CodeLlama-13b reaches 42.1% on HumanEval, 3.7 points above the best individual model (38.4%, token-level). On MBPP the ensemble matches the best byte-level member (53.6%). These gains come without any training, vocabulary alignment or model modification.

Limitation: divergent members. Ensembling does not help when the members differ too much in capability. On GSM8K, Yi-1.5-6B outperforms Mistral-7B-v0.3 by more than 20 points (61.5 vs. 39.0), and the uniform ensemble (55.8) falls below its best member. Uniform averaging lets the weaker model's errors corrupt multi-step reasoning chains, where a single wrong byte can derail the answer. Weighted or adaptive combination rules, which our framework supports directly, are a natural remedy that we leave to future work.

5 Conclusion

We identified *tokenization bias*: the predictive distribution of a tokenized LM depends on how the prompt is tokenized, not only on its bytes, and this dependence persists even for a perfect model. We showed that it can be removed exactly. The Byte-Token Representation Lemma expresses byte-level probabilities as sums over a small set of valid cover encodings. A backward search finds these encodings, and an incremental sampler produces next-byte predictions with one model call per byte. The result is an inference-time procedure that turns any tokenized LM into a statistically equivalent token-free LM. Empirically, it recovers true transition probabilities on Markov chains. It largely eliminates the mid-token failure in FIM code completion, with an improvement of about 18 points in the SPM format, and outperforms token healing and token alignment. It also enables direct ensembling of LMs with different vocabularies, which improves over the best single model on most benchmarks we tested.

Limitations and future work. Our guarantees rely on prefix consistency (Assumption 1), which holds exactly for MPE and approximately for BPE. They also treat the trained LM as an exact model of the token process. Real models deviate from it, which we handle by renormalization. Byte-level generation makes one model call per byte instead of per token, so generation costs more model calls than token-level decoding. Hybrid schemes that switch to byte-level prediction only near problematic boundaries could reduce this overhead. On some generation benchmarks, notably HumanEval, byte-level decoding of single models underperforms token-level decoding, and the causes deserve further study. Finally, uniform averaging fails when members differ greatly in quality. Learned weights, mixtures of experts and byte-level combination with speculative decoding are promising directions.

Reproducibility Statement

Section 3 specifies all definitions, assumptions and algorithms, and Appendix A contains complete proofs. The method requires no training and uses only the public tokenizers and weights of the evaluated models. All benchmarks are publicly available, and Section 4 describes the evaluation protocols and baselines.

References

Ben Athiwaratkun, Shiqi Wang, Mingyue Shang, Yuchen Tian, Zijian Wang, Sujan Kumar Gonugondla, Sanjay Krishna Gouda, Robert Kwiatkowski, Ramesh Nallapati, Parminder Bhatia, and Bing Xiang. Token alignment via character matching for subword completion. In *Findings of the Association for Computational Linguistics: ACL 2024*, 2024.

Jacob Austin, Augustus Odena, Maxwell Nye, Maarten Bosma, Henryk Michalewski, David Dohan, Ellen Jiang, Carrie Cai, Michael Terry, Quoc Le, et al. Program synthesis with large language models. *arXiv preprint arXiv:2108.07732*, 2021.

Mohammad Bavarian, Heewoo Jun, Nikolas Tezak, John Schulman, Christine McLeavey, Jerry Tworek, and Mark Chen. Efficient training of language models to fill in the middle. *arXiv preprint arXiv:2207.14255*, 2022.

Yonatan Bisk, Rowan Zellers, Ronan Le Bras, Jianfeng Gao, and Yejin Choi. PIQA: Reasoning about physical commonsense in natural language. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 34, pp. 7432–7439, 2020.

Kaj Bostrom and Greg Durrett. Byte pair encoding is suboptimal for language model pretraining. In *Findings of the Association for Computational Linguistics: EMNLP 2020*, pp. 4617–4624, 2020.

Tom Brown, Benjamin Mann, Nick Ryder, Melanie Subbiah, Jared D. Kaplan, Prafulla Dhariwal, Arvind Neelakantan, Pranav Shyam, Girish Sastry, Amanda Askell, et al. Language models are few-shot learners. In *Advances in Neural Information Processing Systems*, volume 33, pp. 1877–1901, 2020.

Kris Cao and Laura Rimell. You should evaluate your language model on marginal likelihood over tokenisations. In *Proceedings of the 2021 Conference on Empirical Methods in Natural Language Processing*, pp. 2104–2114, 2021.

Mark Chen, Jerry Tworek, Heewoo Jun, Qiming Yuan, Henrique Ponde de Oliveira Pinto, Jared Kaplan, Harri Edwards, Yuri Burda, Nicholas Joseph, Greg Brockman, et al. Evaluating large language models trained on code. *arXiv preprint arXiv:2107.03374*, 2021.

Jonathan H. Clark, Dan Garrette, Iulia Turc, and John Wieting. CANINE: Pre-training an efficient tokenization-free encoder for language representation. *Transactions of the Association for Computational Linguistics*, 10:73–91, 2022.

Karl Cobbe, Vineet Kosaraju, Mohammad Bavarian, Mark Chen, Heewoo Jun, Lukasz Kaiser, Matthias Plappert, Jerry Tworek, Jacob Hilton, Reiichiro Nakano, et al. Training verifiers to solve math word problems. *arXiv preprint arXiv:2110.14168*, 2021.

Gautier Dagan, Gabriel Synnaeve, and Baptiste Rozière. Getting the most out of your tokenizer for pre-training and domain adaptation. In *Proceedings of the 41st International Conference on Machine Learning*, 2024.

Thomas G. Dietterich. Ensemble methods in machine learning. In *International Workshop on Multiple Classifier Systems*, pp. 1–15, 2000.

Daniel Fried, Armen Aghajanyan, Jessy Lin, Sida Wang, Eric Wallace, Freda Shi, Ruiqi Zhong, Wen-tau Yih, Luke Zettlemoyer, and Mike Lewis. InCoder: A generative model for code infilling and synthesis. In *International Conference on Learning Representations*, 2023.

Juan Luis Gastaldi, John Terilla, Luca Malagutti, Brian DuSell, Tim Vieira, and Ryan Cotterell. The foundations of tokenization: Statistical and computational concerns. *arXiv preprint arXiv:2407.11606*, 2024.

Dan Hendrycks, Collin Burns, Steven Basart, Andy Zou, Mantas Mazeika, Dawn Song, and Jacob Steinhardt. Measuring massive multitask language understanding. In *International Conference on Learning Representations*, 2021.

Yichong Huang, Xiaocheng Feng, Baohang Li, Yang Xiang, Hui Wang, Bing Qin, and Ting Liu. Ensemble learning for heterogeneous large language models with deep parallel collaboration. *arXiv preprint arXiv:2404.12715*, 2024.

Robert A. Jacobs, Michael I. Jordan, Steven J. Nowlan, and Geoffrey E. Hinton. Adaptive mixtures of local experts. *Neural Computation*, 3(1):79–87, 1991.

Albert Q. Jiang, Alexandre Sablayrolles, Arthur Mensch, Chris Bamford, Devendra Singh Chaplot, Diego de las Casas, Florian Bressand, Gianna Lengyel, Guillaume Lample, Lucile Saulnier, et al. Mistral 7B. *arXiv preprint arXiv:2310.06825*, 2023a.

Dongfu Jiang, Xiang Ren, and Bill Yuchen Lin. LLM-Blender: Ensembling large language models with pairwise ranking and generative fusion. In *Proceedings of the 61st Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pp. 14165–14178, 2023b.

Mandar Joshi, Eunsol Choi, Daniel S. Weld, and Luke Zettlemoyer. TriviaQA: A large scale distantly supervised challenge dataset for reading comprehension. In *Proceedings of the 55th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pp. 1601–1611, 2017.

Taku Kudo. Subword regularization: Improving neural network translation models with multiple subword candidates. In *Proceedings of the 56th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pp. 66–75, 2018.

Taku Kudo and John Richardson. SentencePiece: A simple and language independent subword tokenizer and detokenizer for neural text processing. In *Proceedings of the 2018 Conference on Empirical Methods in Natural Language Processing: System Demonstrations*, pp. 66–71, 2018.

Tom Kwiatkowski, Jennimaria Palomaki, Olivia Redfield, Michael Collins, Ankur Parikh, Chris Alberti, Danielle Epstein, Illia Polosukhin, Jacob Devlin, Kenton Lee, et al. Natural questions: A benchmark for question answering research. *Transactions of the Association for Computational Linguistics*, 7:452–466, 2019.

Sabrina J. Mielke, Zaid Alyafeai, Elizabeth Salesky, Colin Raffel, Manan Dey, Matthias Gallé, Arun Raja, Chenglei Si, Wilson Y. Lee, Benoît Sagot, and Samson Tan. Between words and characters: A brief history of open-vocabulary modeling and tokenization in NLP. *arXiv preprint arXiv:2112.10508*, 2021.

Benjamin Minixhofer, Edoardo Maria Ponti, and Ivan Vulić. Zero-shot tokenizer transfer. *arXiv preprint arXiv:2405.07883*, 2024.

Byung-Doh Oh and William Schuler. Leading whitespaces of language models' subword vocabulary pose a confound for calculating word probabilities. *arXiv preprint arXiv:2406.10851*, 2024.

Aleksandar Petrov, Emanuele La Malfa, Philip H. S. Torr, and Adel Bibi. Language model tokenizers introduce unfairness between languages. In *Advances in Neural Information Processing Systems*, volume 36, 2023.

Tiago Pimentel and Clara Meister. How to compute the probability of a word. In *Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing*, 2024.

Ivan Provilkov, Dmitrii Emelianenko, and Elena Voita. BPE-dropout: Simple and effective subword regularization. In *Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics*, pp. 1882–1892, 2020.

Alec Radford, Jeffrey Wu, Rewon Child, David Luan, Dario Amodei, and Ilya Sutskever. Language models are unsupervised multitask learners. Technical report, OpenAI, 2019.

Nived Rajaraman, Jiantao Jiao, and Kannan Ramchandran. Toward a theory of tokenization in LLMs. *arXiv preprint arXiv:2404.08335*, 2024.

Baptiste Rozière, Jonas Gehring, Fabian Gloeckle, Sten Sootla, Itai Gat, Xiaoqing Ellen Tan, Yossi Adi, Jingyu Liu, Tal Remez, Jérémy Rapin, et al. Code Llama: Open foundation models for code. *arXiv preprint arXiv:2308.12950*, 2023.

Phillip Rust, Jonas Pfeiffer, Ivan Vulić, Sebastian Ruder, and Iryna Gurevych. How good is your tokenizer? on the monolingual performance of multilingual language models. In *Proceedings of the 59th Annual Meeting of the Association for Computational Linguistics and the 11th International Joint Conference on Natural Language Processing (Volume 1: Long Papers)*, pp. 3118–3135, 2021.

Mike Schuster and Kaisuke Nakajima. Japanese and Korean voice search. In *IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)*, pp. 5149–5152, 2012.

Rico Sennrich, Barry Haddow, and Alexandra Birch. Neural machine translation of rare words with subword units. In *Proceedings of the 54th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pp. 1715–1725, 2016.

Noam Shazeer, Azalia Mirhoseini, Krzysztof Maziarz, Andy Davis, Quoc Le, Geoffrey Hinton, and Jeff Dean. Outrageously large neural networks: The sparsely-gated mixture-of-experts layer. In *International Conference on Learning Representations*, 2017.

Yi Tay, Vinh Q. Tran, Sebastian Ruder, Jai Gupta, Hyung Won Chung, Dara Bahri, Zhen Qin, Simon Baumgartner, Cong Yu, and Donald Metzler. Charformer: Fast character transformers via gradient-based subword tokenization. In *International Conference on Learning Representations*, 2022.

Hugo Touvron, Louis Martin, Kevin Stone, Peter Albert, Amjad Almahairi, Yasmine Babaei, Nikolay Bashlykov, Soumya Batra, Prajjwal Bhargava, Shruti Bhosale, et al. Llama 2: Open foundation and fine-tuned chat models. *arXiv preprint arXiv:2307.09288*, 2023.

Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Łukasz Kaiser, and Illia Polosukhin. Attention is all you need. In *Advances in Neural Information Processing Systems*, volume 30, 2017.

Fanqi Wan, Xinting Huang, Deng Cai, Xiaojun Quan, Wei Bi, and Shuming Shi. Knowledge fusion of large language models. In *International Conference on Learning Representations*, 2024.

Junxiong Wang, Tushaar Gangavarapu, Jing Nathan Yan, and Alexander M. Rush. MambaByte: Token-free selective state space model. In *First Conference on Language Modeling*, 2024.

Xuezhi Wang, Jason Wei, Dale Schuurmans, Quoc Le, Ed Chi, Sharan Narang, Aakanksha Chowdhery, and Denny Zhou. Self-consistency improves chain of thought reasoning in language models. In *International Conference on Learning Representations*, 2023.

Yangyifan Xu, Jinliang Lu, and Jiajun Zhang. Bridging the gap between different vocabularies for LLM ensemble. In *Proceedings of the 2024 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies (Volume 1: Long Papers)*, pp. 7140–7152, 2024.

Linting Xue, Aditya Barua, Noah Constant, Rami Al-Rfou, Sharan Narang, Mihir Kale, Adam Roberts, and Colin Raffel. ByT5: Towards a token-free future with pre-trained byte-to-byte models. *Transactions of the Association for Computational Linguistics*, 10:291–306, 2022.

Alex Young, Bei Chen, Chao Li, Chengen Huang, Ge Zhang, Guanwei Zhang, Heng Li, Jiangcheng Zhu, Jianqun Chen, Jing Chang, et al. Yi: Open foundation models by 01.AI. *arXiv preprint arXiv:2403.04652*, 2024.

Lili Yu, Dániel Simig, Colin Flaherty, Armen Aghajanyan, Luke Zettlemoyer, and Mike Lewis. MEGABYTE: Predicting million-byte sequences with multiscale transformers. In *Advances in Neural Information Processing Systems*, volume 36, 2023.

Yao-Ching Yu, Chun-Chih Kuo, Ziqi Ye, Yu-Cheng Chang, and Yueh-Se Li. Breaking the ceiling of the LLM community by treating token generation as a classification for ensembling. *arXiv preprint arXiv:2406.12585*, 2024.

Vilém Zouhar, Clara Meister, Juan Luis Gastaldi, Li Du, Mrinmaya Sachan, and Ryan Cotterell. Tokenization and the noiseless channel. In *Proceedings of the 61st Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pp. 5184–5207, 2023.

A PROOFS

Proof of Lemma 1. Let $E = \{\mathrm{X}_1^n = x_1^n\}$ and, for $\boldsymbol{t} \in \mathrm{cover}(x_1^n)$, let $E_{\boldsymbol{t}} = \{\mathrm{T}_1^{|\boldsymbol{t}|} = \boldsymbol{t}\}$. Then $\Pr(E_{\boldsymbol{t}}) = P(\boldsymbol{t})$. We show that the sets $E_{\boldsymbol{t}}$ are pairwise disjoint and that their union is E.

(Union $\supseteq E$.) Take an outcome in E. The tokens of $\mathrm{T} = \mathrm{encode}(\mathrm{X})$ concatenate to X, so there is a unique k such that $|\,\mathrm{decode}(\mathrm{T}_1^{k-1})| < n \le |\,\mathrm{decode}(\mathrm{T}_1^{k})|$. Since X begins with x_1^n, the string x_1^n is a prefix of $\mathrm{decode}(\mathrm{T}_1^k)$, and $\mathrm{decode}(\mathrm{T}_1^{k-1})$ is a proper prefix of x_1^n. By Assumption 1, T_1^k is valid. Hence $\mathrm{T}_1^k \in \mathrm{cover}(x_1^n)$, and the outcome lies in $E_{\mathrm{T}_1^k}$.

(Union $\subseteq E$.) If $\mathrm{T}_1^{|\boldsymbol{t}|} = \boldsymbol{t}$ for some $\boldsymbol{t} \in \mathrm{cover}(x_1^n)$, then X begins with $\mathrm{decode}(\boldsymbol{t})$, which has x_1^n as a prefix.

(Disjointness.) Suppose $\boldsymbol{t} = t_1^k$ and $\boldsymbol{t}' = t_1'^{k'}$ are distinct cover encodings that are both prefixes of T. Then one is a proper prefix of the other; say $k < k'$, so $\boldsymbol{t}$ is a prefix of $t_1'^{k'-1}$. This gives $n \le |\,\mathrm{decode}(\boldsymbol{t})| \le |\,\mathrm{decode}(t_1'^{k'-1})| < n$, a contradiction.

Therefore $P(x_1^n) = \Pr(E) = \sum_{\boldsymbol{t}\in\mathrm{cover}(x_1^n)} \Pr(E_{\boldsymbol{t}}) = \sum_{\boldsymbol{t}\in\mathrm{cover}(x_1^n)} P(\boldsymbol{t})$. □

Proof of Proposition 2. Let $\boldsymbol{t} = t_1^k \in \mathrm{cover}(x_1^n a)$, and let i be the start position of t_k, i.e., $i - 1 = |\,\mathrm{decode}(t_1^{k-1})|$, with $i \le n+1$. *Case $i \le n$:* then $\mathrm{decode}(t_1^{k-1})$ is a proper prefix of x_1^n, and $\mathrm{decode}(\boldsymbol{t})$ has $x_1^n a$, hence x_1^n, as a prefix. So $\boldsymbol{t} \in \mathrm{cover}(x_1^n)$, $|\,\mathrm{decode}(\boldsymbol{t})| > n$ and $\mathrm{decode}(\boldsymbol{t})_{n+1} = a$, i.e., $\boldsymbol{t} \in \mathcal{C}_{n+1}(a)$. *Case $i = n+1$:* by Lemma 2, $t_1^{k-1} = \mathrm{encode}(x_1^n) = \boldsymbol{\tau}_n$, and t_k begins with a. Since $\boldsymbol{t}$ is valid, $\boldsymbol{t} \in \bar{\mathcal{C}}_{n+1}(a)$. Conversely, each element of $\mathcal{C}_{n+1}(a)$ is valid, has $x_1^n a$ as a prefix of its decoding, and its history decodes to a proper prefix of x_1^n, hence of $x_1^n a$. Each element of $\bar{\mathcal{C}}_{n+1}(a)$ is valid, decodes to $x_1^n v$ with $v_1 = a$, and its history decodes to x_1^n, a proper prefix of $x_1^n a$. Both sets are thus contained in $\mathrm{cover}(x_1^n a)$. They are disjoint because their last tokens start at different positions. Eq. (3) follows from Lemma 1 and $P(\boldsymbol{\tau}_n \oplus v) = P(\boldsymbol{\tau}_n)P(v \mid \boldsymbol{\tau}_n)$. □

B ADDITIONAL IMPLEMENTATION NOTES

Pseudocode. Algorithm 2 gives the complete next-byte sampling procedure of Section 3.5.

Cost per generated byte. Algorithm 2 needs one forward pass of the LM per generated byte, on the context $\mathrm{encode}(x_1^n)$. Consecutive contexts $\mathrm{encode}(x_1^n)$ and $\mathrm{encode}(x_1^{n+1})$ share all tokens except possibly the last few, so key-value caches can be reused for the common prefix. Validity checks only involve the tokenizer. They can be restricted to candidate tokens v with non-negligible $P(v \mid \boldsymbol{\tau}_n)$, and since the last token of a cover encoding has length at most L, at most L bytes of a token remain pending at any time. The size of the maintained cover set is bounded as in Lemma 2.

Ensembles. For an ensemble of M models, each member runs Algorithm 2 with its own tokenizer and cover set. At each step the byte distributions $\{P_m(\cdot \mid x_1^n)\}_{m=1}^M$ are averaged, a byte is chosen from the average, and the sampled byte is fed back to all members, each of which updates its own cover set. Since all members share the same byte history, no alignment between vocabularies is needed.

Algorithm 2 Exact next-byte sampling from a tokenized LM

Require: prompt x_1^n, tokenized LM P, number of bytes m

$\mathcal{C} \leftarrow \textsc{BackwardSearch}(x_1^n)$ ▷ Algorithm 1, run once
for $s = 1, \ldots, m$ **do**
  $\boldsymbol{\tau} \leftarrow \mathrm{encode}(x_1^n)$; compute $P(\cdot \mid \boldsymbol{\tau})$ ▷ one model call; $P(\boldsymbol{\tau})$ is stored in $\mathcal{C}$
  for each $a \in \mathcal{A}$ **do**
    $\mathcal{C}_{n+1}(a) \leftarrow \{(\boldsymbol{t}, p) \in \mathcal{C} : |\,\mathrm{decode}(\boldsymbol{t})| > n,\ \mathrm{decode}(\boldsymbol{t})_{n+1} = a\}$ ▷ regroup
    $\bar{\mathcal{C}}_{n+1}(a) \leftarrow \{(\boldsymbol{\tau} \oplus v,\ P(\boldsymbol{\tau})P(v \mid \boldsymbol{\tau})) : v_1 = a,\ \boldsymbol{\tau} \oplus v \text{ valid}\}$ ▷ extend
    $w_a \leftarrow \sum_{(\boldsymbol{t},p) \in \mathcal{C}_{n+1}(a) \cup \bar{\mathcal{C}}_{n+1}(a)} p$ ▷ $= P(x_1^n a)$ by Eq. (3)
  end for
  sample $x_{n+1} \sim w_a / \sum_{a'} w_{a'}$ (or take $\arg\max_a w_a$ for greedy decoding)
  $\mathcal{C} \leftarrow \mathcal{C}_{n+1}(x_{n+1}) \cup \bar{\mathcal{C}}_{n+1}(x_{n+1})$; $n \leftarrow n + 1$
end for
return x_1^n

Relation to token healing and alignment. Token healing removes the last prompt token and constrains the next token to start with its bytes. This corresponds to keeping only the extension group with i equal to the start of the last prompt token in Algorithm 1, without validity checks and without the remaining cover encodings. Token alignment backs off several tokens, and so still conditions on one tokenization of the shortened prompt. Both therefore approximate, but do not compute, the sum in Eq. (2).